

Lightweight Trypanosome Parasite Detection Using YOLOv7-Tiny

Anna Park
06/01/2026
Yilmaz – Period 7

Table of Contents

Abstract.....	3
I. Introduction.....	4
II. Background.....	4
III. Applications.....	7
IV. Methods.....	8
V. Results.....	14
VI. Limitations.....	18
VII. Conclusion.....	18
VIII. Future Work and Recommendations.....	19
IX. Materials.....	19
References.....	20
Appendix.....	21

Abstract

Human African trypanosomiasis (HAT), commonly known as sleeping sickness, remains a significant public health threat in sub-Saharan Africa, where it is almost always fatal without treatment. While case numbers have declined due to active surveillance programs, diagnosis still depends on skilled microscopists manually examining blood smears — a labor-intensive process that is impractical in remote, resource-limited settings where the disease is most prevalent. Existing deep learning approaches for trypanosome detection have demonstrated strong accuracy, but typically rely on GPU-equipped hardware and complex multi-model pipelines that are inaccessible in the field clinics where they are most needed. This project addresses that gap by training a YOLOv7-tiny object detection model to identify *Trypanosoma brucei brucei* parasites in microscopy images of unstained thick blood smears. Using the publicly available Tryp dataset, the model was fine-tuned from COCO-pretrained weights using only free cloud computing resources (Google Colab T4 GPU). After 100 epochs of training, the model achieved 74.1% AP@0.5, 74.9% precision, and 70.2% recall on the test set. The trained model was exported to ONNX format, producing a 12 MB file capable of running inference on a standard laptop CPU without any GPU hardware. A standalone inference script and a web-based interface were also developed to demonstrate practical deployment. These results suggest that competitive trypanosome detection is achievable with lightweight models on low-cost hardware, offering a viable path toward more accessible diagnostic screening tools in endemic regions.

I. Introduction

Human African trypanosomiasis (HAT), caused by the *Trypanosoma brucei* parasite and transmitted through tsetse fly bites, is a neglected tropical disease that is nearly 100% fatal if untreated [1]. While sustained control efforts have reduced the number of reported cases dramatically — from nearly 28,000 in 1999 to under 600 in 2024 — the disease continues to affect populations in remote sub-Saharan Africa where health infrastructure is weak.

Approximately 55 million people remain at risk of infection, and the decline in cases is directly tied to active surveillance programs that depend on trained personnel performing microscopic examination of blood smears [2].

This reliance on skilled microscopists presents a fundamental bottleneck. Microscopic diagnosis is labor-intensive, subjective, and error-prone, particularly during chronic infections when parasite levels are low. In the rural areas where HAT is endemic, trained laboratory technicians are scarce, and the equipment and expertise required for accurate diagnosis are often unavailable [3]. A missed diagnosis means a missed opportunity for treatment, and the disease progresses to central nervous system involvement and eventual death.

This project was motivated by the question: can we build an automated parasite detection tool that is accurate enough to be clinically useful, yet lightweight enough to run on the kind of hardware actually available in resource-limited settings — specifically, a basic laptop with no dedicated GPU?

The approach taken was to train YOLOv7-tiny [4], a compact object detection model with approximately 6 million parameters, on the Tryp dataset of microscopy images. The novelty of this project lies not in the model architecture itself, but in demonstrating a complete, accessible pipeline: training was performed entirely on free cloud resources (Google Colab), the resulting model was exported to ONNX format for efficient CPU inference, and a standalone detection tool was built that requires only three Python packages to operate. The entire model is approximately 12 MB in size — small enough to distribute via email or USB drive. This stands in contrast to existing approaches that rely on large models, GPU hardware, or proprietary platforms, all of which limit practical deployment in the settings where trypanosome screening is most needed.

II. Background

The Disease

Human African trypanosomiasis is caused by two subspecies of the *Trypanosoma brucei* parasite: *T. b. gambiense*, which accounts for over 92% of reported cases and causes a chronic illness that can persist for months or years before symptoms emerge, and *T. b. rhodesiense*, which causes a rapidly progressing acute form of the disease [1]. Both forms are transmitted through the bite of the tsetse fly and are found exclusively in sub-Saharan Africa. Without treatment, HAT is considered uniformly fatal, as the parasite eventually crosses the blood-brain

barrier and invades the central nervous system, causing the sleep disturbances that give the disease its common name.

The current standard for diagnosis is microscopic examination of blood smears, typically Giemsa-stained thin films or unstained thick films. Thick blood smears are often preferred for initial screening because they concentrate more blood per field of view, improving the likelihood of spotting parasites when levels are low. However, thick smears also present more visual noise and complexity, making automated analysis more challenging.

Existing Approaches

Several research groups have applied deep learning to trypanosome detection. Kittichai et al. [5] developed a hybrid deep learning system using a two-stage pipeline — first detecting parasites with an object detection model, then classifying them by species (*T. cruzi*, *T. brucei*, and *T. evansi*) — running on a custom low-code AI platform called CiRA CORE. Their approach achieved high accuracy in species classification across three trypanosome types using oil-immersion microscopy images of Giemsa-stained thin films. However, the system requires specialized software and GPU hardware.

Anzaku et al. [6] published the Tryp dataset and benchmarked several detection models including Faster R-CNN, RetinaNet, and YOLOv7 on unstained thick blood smears. Their work established baseline detection performance on this challenging dataset and demonstrated that YOLO-family architectures could be applied to trypanosome detection, though their focus was not on lightweight deployment.

More recently, Anzaku et al. [7] proposed YOLO-Tryp, a modified YOLO architecture that added a P2 detection head specifically to improve localization of small parasites, achieving 71.3% AP@0.5 on the Tryp test set.

The Tryp Dataset

This project uses the Tryp dataset published by Anzaku et al. [6], which is publicly available on Figshare. The dataset contains microscopy images of unstained thick blood smears from mice infected with *Trypanosoma brucei brucei*. The images were captured from microscope video recordings, with individual frames extracted and annotated with bounding boxes around visible parasites. A representative sample image is shown in Fig. 1, and a sample image with its corresponding bounding box annotations is shown in Fig. 2.

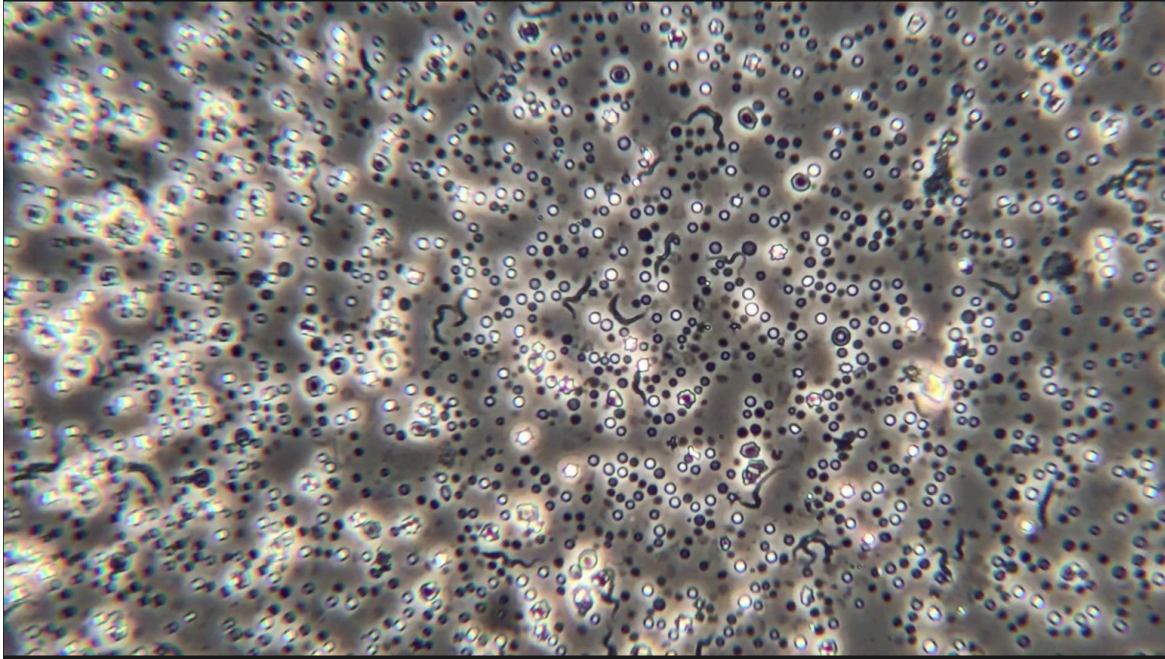


Fig. 1. Example unstained thick blood smear image from the Tryp dataset showing the appearance of *Trypanosoma brucei brucei* parasites among red blood cells. [6]

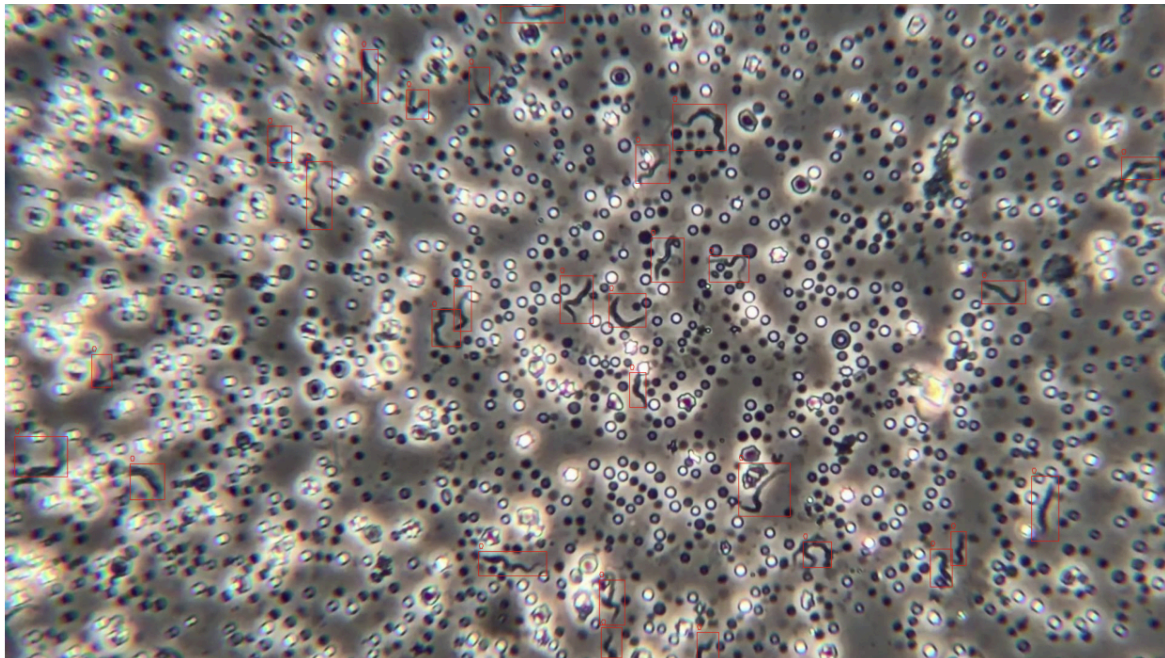


Fig. 2. The same type of image as in Fig. 1, with ground-truth bounding boxes overlaid on annotated parasite locations. [6]

The dataset used in this project is organized as follows:

- Training set: 984 images

- Validation set: 629 images
- Test set: 631 images

Each image is accompanied by a label file in YOLO format, containing one line per annotated parasite with the class ID (0, since there is only one class) and normalized bounding box coordinates (center x, center y, width, height). The test set contains a total of approximately 9,094 parasite annotations across 631 images, with an average of roughly 14 parasites per image.

A key characteristic of this dataset is the use of unstained thick blood smears, which are more practical for field use than stained thin films but present greater challenges for automated detection due to increased visual noise and the absence of staining contrast.

Tools and Frameworks

- **YOLOv7**: The base detection framework, developed by Wang et al. [4], which introduced efficient layer aggregation (E-ELAN) and model scaling techniques.
- **PyTorch**: The deep learning framework used for model training.
- **Google Colab**: Free cloud computing platform providing T4 GPU access for training.
- **ONNX / ONNX Runtime [8]**: Open format for model export and optimized CPU inference.
- **OpenCV**: Image processing library used in the inference pipeline.
- **Streamlit**: Python framework used to build the web-based detection interface.

III. Applications

The primary intended application of this work is automated screening of blood smear images for trypanosome parasites in clinical settings where trained microscopists are unavailable. This is especially useful for field clinics in endemic regions. In rural sub-Saharan Africa, where HAT is most prevalent, health facilities often lack the personnel and equipment for traditional microscopic diagnosis. A detection tool that runs on a standard laptop could enable community health workers to screen blood smears and flag potentially positive samples for confirmation by a specialist, even in locations without internet connectivity or specialized hardware.

This project can also be used for surveillance and epidemiological monitoring. The WHO's 2030 roadmap targets elimination of gambiense HAT transmission [9]. Maintaining surveillance in formerly endemic areas requires ongoing screening, which becomes increasingly expensive per case as prevalence drops. An automated screening tool could reduce the cost and personnel requirements of active surveillance programs.

Some other uses include veterinary applications, training aides, and research. Trypanosomiasis also affects livestock across sub-Saharan Africa, causing significant economic losses. The same detection approach could be adapted for veterinary screening of animal blood samples. In addition to this, an automated detection system could serve as a training aid for new

microscopists or as a quality control tool, providing a second opinion on slides that have been manually examined. Finally, automated counting and localization of parasites in blood smears could accelerate research workflows in laboratory settings studying trypanosome biology, drug efficacy, or vaccine development.

IV. Methods

System Architecture

The complete detection pipeline takes a single microscopy image as input and produces an annotated output image with bounding boxes drawn around detected parasites and corresponding confidence scores. The end-to-end system architecture is shown in Fig. 3.

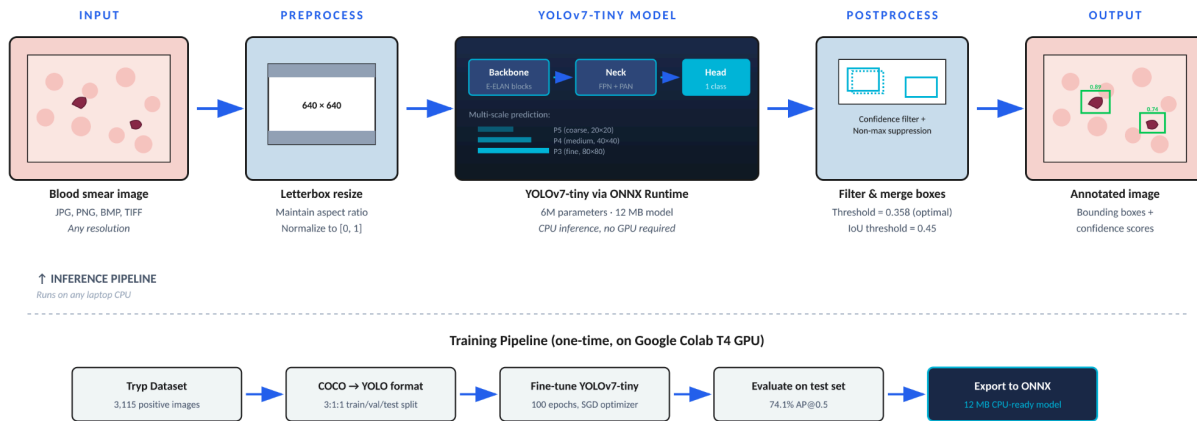


Fig. 3. System architecture diagram showing the end-to-end detection pipeline, from raw blood smear input through preprocessing, model inference, postprocessing, and final annotated output.

Model Architecture

This project uses YOLOv7-tiny, a compact variant of the YOLOv7 [4] object detection architecture. The model is descended from the original YOLO design [10] and consists of three main components: a backbone, a neck (FPN+PAN), and a detection head. Fig. 4 shows the detailed neural network architecture.

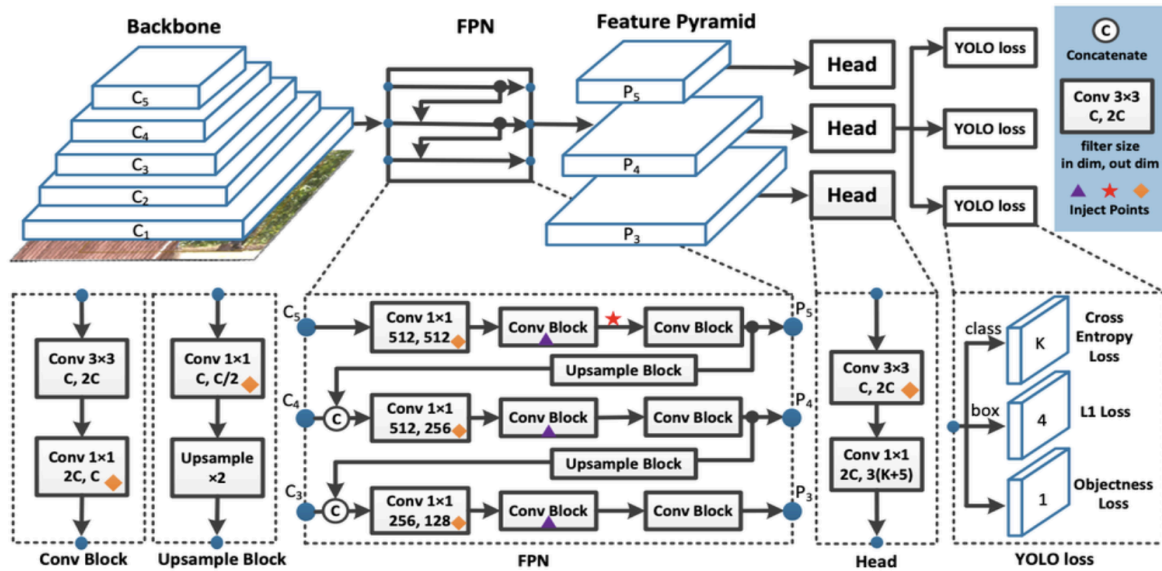


Fig. 4. YOLOv7-tiny architecture. The backbone produces multi-scale feature maps (C_1 – C_5), the FPN+PAN neck fuses these features to produce the P_3 – P_5 feature pyramid, and three detection heads predict bounding boxes, objectness scores, and class probabilities at each scale. Source: [11]

Backbone. The backbone uses lightweight E-ELAN (Extended Efficient Layer Aggregation Network) blocks to extract multi-scale feature maps from the input image. These blocks use a combination of 3×3 and 1×1 convolutions with efficient channel aggregation to minimize parameter count while preserving representational capacity. The backbone produces feature maps at multiple scales (C_3 , C_4 , C_5), capturing both fine-grained local features and higher-level contextual patterns.

Neck (FPN + PAN). The Feature Pyramid Network (FPN) with Path Aggregation Network (PAN) fuses features across the multiple scales produced by the backbone. High-level features from C_5 are progressively combined with finer-grained features from C_4 and C_3 through upsampling and concatenation operations. This multi-scale fusion is critical for this task because trypanosome parasites are small relative to the full image, requiring the model to combine fine spatial detail from lower feature levels with semantic understanding from higher levels. The neck produces the feature pyramid (P_3 , P_4 , P_5) that feeds into the detection heads.

Detection Head. Each level of the feature pyramid has a detection head that predicts bounding box coordinates, objectness scores, and class probabilities. For this project, the detection head was adapted from the default 80-class COCO configuration to a single class (trypanosome). At each spatial location in the feature map, the model predicts whether a parasite is present and, if so, where it is located. The final predictions are produced by combining outputs across all three detection scales after non-maximum suppression to eliminate duplicate detections.

Model specifications:

- Parameters: ~6 million (compared to ~37 million for full YOLOv7)
- GFLOPs: 13.0
- Model size: ~12 MB (ONNX)
- Input resolution: 640 × 640

Understanding Grid-Based Detection

It is important to understand how YOLO detection works to properly interpret the evaluation metrics. YOLOv7 does not classify entire images as positive or negative. Instead, it divides each 640×640 input image into a grid of cells across three detection scales. Each cell independently predicts whether it contains a parasite. This grid-based prediction process is illustrated conceptually in Fig. 5.

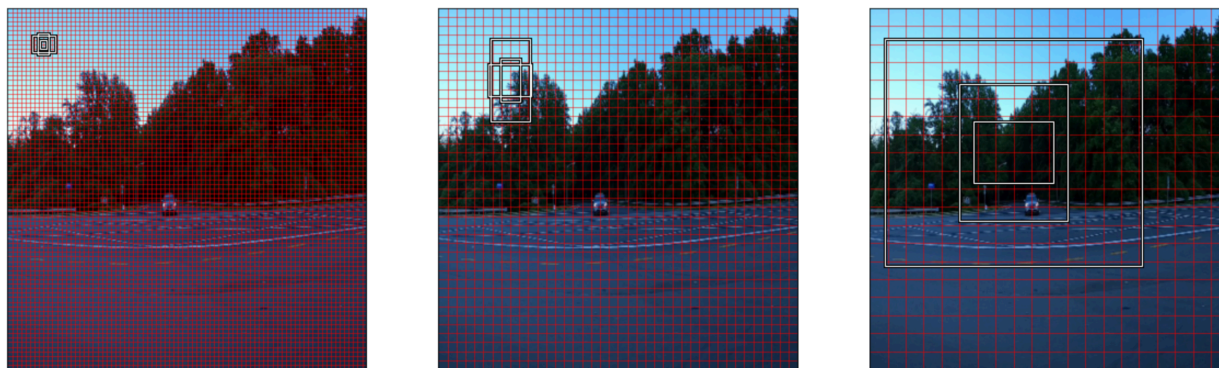


Fig. 5. Conceptual visualization of YOLO's grid-based detection across three scales. Each grid cell makes an independent prediction, and only a small fraction of cells correspond to actual parasite locations. Source: [12]

In a typical blood smear image from this dataset, only a small number of grid cells actually contain parasites (corresponding to roughly 5–15 bounding box annotations per image), while thousands of cells cover background regions of blood, noise, and empty space. This means the detection task is inherently imbalanced — the vast majority of predictions are on negative regions. This is why raw accuracy would be misleading as a metric, and why precision, recall, and mean average precision (mAP) are used instead to evaluate detection performance.

Detection Pipeline (Pseudocode)

The inference pipeline implemented in the standalone CLI tool and the Streamlit interface can be summarized as follows:

INPUT: image (any resolution), confidence threshold *conf*, IoU threshold *iou*

OUTPUT: list of (*x1*, *y1*, *x2*, *y2*, confidence) bounding boxes

1. *resized* ← letterbox(image, target_size=640)

2. `normalized` $\leftarrow$ `resized / 255.0`
3. `tensor` $\leftarrow$ `transpose(normalized) to (1, 3, 640, 640)`
4. `predictions` $\leftarrow$ `ONNX_model.run(tensor)`
5. `scored` $\leftarrow$ filter predictions where `score > conf`
6. `boxes` $\leftarrow$ convert center-form to corner-form
7. `kept` $\leftarrow$ `non_max_suppression(boxes, iou_threshold=iou)`
8. `scaled` $\leftarrow$ rescale boxes to original image coordinates
9. RETURN `scaled`

Training Configuration

The model was trained using transfer learning, starting from weights pretrained on the COCO dataset [13], a large-scale object detection dataset containing 80 common object classes. Transfer learning allows the model to leverage general visual features (edges, textures, shapes) learned from COCO and adapt them to the specific task of parasite detection, which is especially valuable given the relatively small size of the Tryp dataset. The complete training configuration is summarized in Table I.

Table I. Training configuration for YOLOv7-tiny on the Tryp dataset.

Parameter	Value
Base weights	COCO pretrained (yolov7-tiny.pt)
Epochs	100
Batch size	16
Image size	640 × 640
Optimizer	SGD (learning rate = 0.01, momentum = 0.937)
Weight decay	0.0005
Augmentation	Mosaic, mixup, horizontal flip, HSV jitter, scale

Parameter	Value
Hardware	Google Colab T4 GPU (free tier)

Training was performed entirely on Google Colab's free tier, using a single NVIDIA T4 GPU. The complete training run of 100 epochs took approximately 2–3 hours. Checkpoints were saved every 10 epochs, with automatic background synchronization to Google Drive every 5 minutes to prevent loss of progress due to session disconnections.

ONNX Export and CPU Deployment

After training, the best-performing checkpoint was exported to ONNX (Open Neural Network Exchange) format [8]. ONNX is an open standard that allows models trained in PyTorch to be run using ONNX Runtime, which provides optimized CPU inference kernels. This export step is central to the project's goal of lightweight deployment: the ONNX model runs significantly faster than raw PyTorch on CPU and requires no GPU or PyTorch installation.

A standalone Python inference script was developed that performs the complete detection pipeline: image loading, letterbox resizing, model inference, confidence filtering, non-maximum suppression, and bounding box visualization. The script requires only three pip packages (onnxruntime, opencv-python, numpy) and runs on any laptop. A web-based interface was also built using Streamlit, allowing users to upload blood smear images through a browser, adjust the confidence threshold, and view detection results with bounding boxes and confidence scores displayed alongside the original image. A screenshot of the deployed Streamlit interface is shown in Fig. 6.

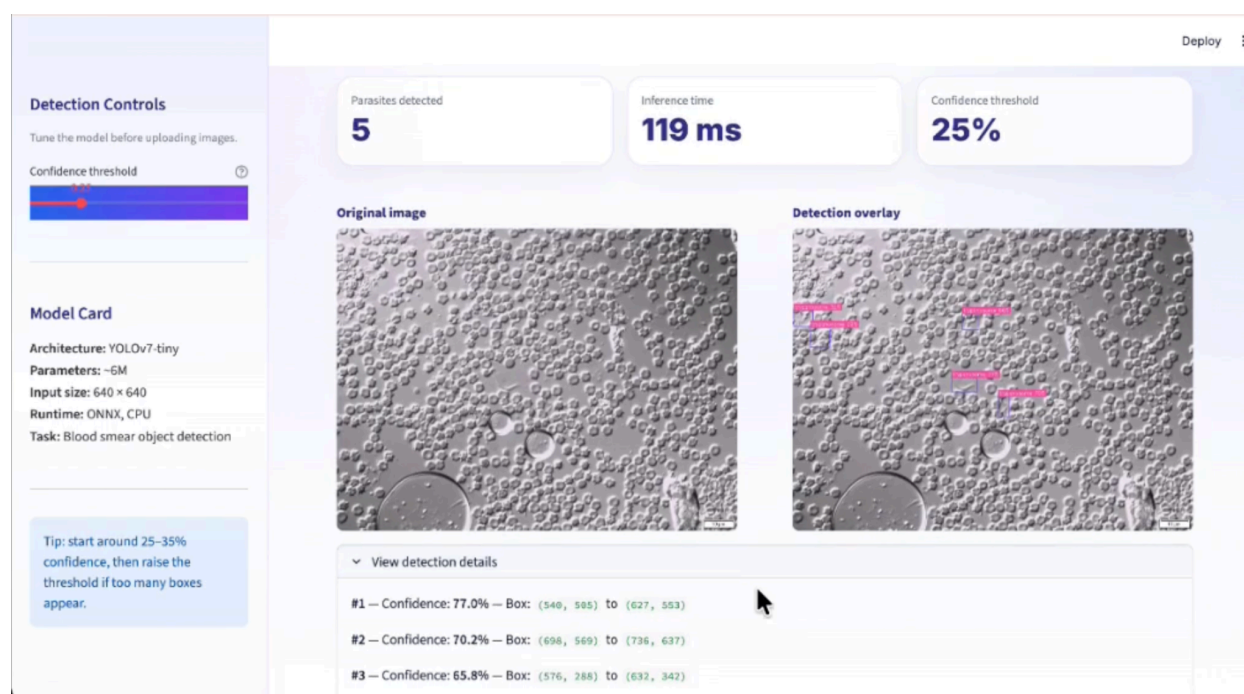


Fig. 6. Screenshot of the deployed Streamlit web interface, displaying side-by-side original and annotated images with detection details and a confidence threshold control.

Evaluation Metrics

The following standard object detection metrics were used to evaluate model performance:

Precision measures the proportion of the model's detections that are correct. A precision of 74.9% means that when the model flags a region as containing a parasite, it is correct about three-quarters of the time. **Recall** measures the proportion of actual parasites that the model successfully detected. A recall of 70.2% means the model finds roughly 7 out of every 10 parasites present in the images. In a diagnostic context, recall is particularly important because a missed parasite represents a missed diagnosis. **AP@0.5** (Average Precision at 50% IoU) is the primary detection metric. It summarizes detection quality across all confidence thresholds, requiring that predicted bounding boxes overlap with ground truth boxes by at least 50% (Intersection over Union ≥ 0.5) to count as correct detections. **AP@0.5:0.95** is a stricter metric that averages AP across multiple IoU thresholds from 0.5 to 0.95, penalizing imprecise bounding box localization more heavily.

Challenges and Iterations

The project went through several iterations and encountered a number of practical obstacles during development. The most significant challenges were not in the model architecture itself but in the surrounding infrastructure required to train, evaluate, and deploy the system reliably.

The first major obstacle was a compatibility issue with PyTorch 2.6+, which changed the default value of `weights_only` in `torch.load` to `True`. This broke YOLOv7's checkpoint loading code, which expected the previous default. A patch was applied at the start of each Colab notebook that injects a wrapper around `torch.load` in every YOLOv7 source file to restore the prior behavior.

Training stability on Google Colab's free tier was another recurring issue. Sessions would disconnect unpredictably, sometimes losing several epochs of training progress. The solution was a background synchronization thread that copies weights from the Colab session to Google Drive every five minutes, combined with checkpoint saves every ten epochs. This made the training pipeline resilient to disconnections.

A third issue arose from naive file transfer. Initially, the preprocessed dataset was loaded by copying thousands of individual files from Google Drive to Colab's local disk, which took over ten minutes due to per-file overhead. Zipping the dataset locally and uploading a single `preprocessed.zip` file directly through Colab's sidebar reduced this step to under a minute.

If more time were available, several extensions would have been worth pursuing. The most valuable would have been incorporating the 93 negative images explicitly during training to

suppress false positives more aggressively. A multi-species classification head — even just a binary "brucei vs. cruzi" extension — would have substantially increased the project's clinical relevance. Confidence calibration using a held-out set would have improved the interpretability of the confidence scores produced by the model.

In hindsight, the project would also have benefited from earlier focus on the deployment pipeline rather than the model itself. The lightweight angle is the strongest aspect of this work, and earlier prototyping of the ONNX export and CPU inference path would have allowed more iteration on the user interface and demonstration tools.

V. Results

The model was evaluated on a held-out test set of 631 images containing approximately 9,094 ground-truth parasite annotations. Standard object detection metrics were used to measure performance across multiple operating points.

Detection Performance

The trained YOLOv7-tiny model achieved the results shown in Table II on the test set.

Table II. Detection performance metrics on the held-out test set of 631 images.

Metric	Value
AP@0.5	74.1%
AP@0.5:0.95	34.4%
Precision	74.9%
Recall	70.2%
Peak F1	0.72 (at confidence threshold 0.358)

The precision-recall curve (Fig. 7) shows the model maintaining high precision (above 0.8) for recall values up to roughly 0.6, with performance degrading gracefully as recall increases. The area under this curve corresponds to the AP@0.5 of 74.1%. F1 analysis (Fig. 8) identified an optimal operating confidence threshold of 0.358, where the model achieves its peak balanced score of 0.72.

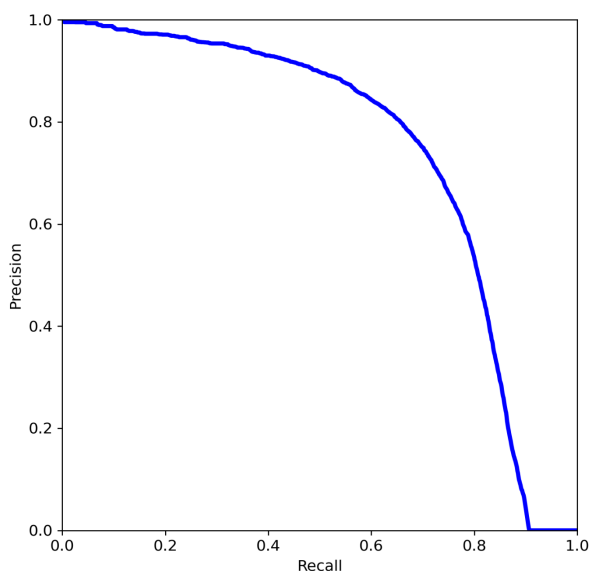


Fig. 7. Precision-recall curve on the test set.

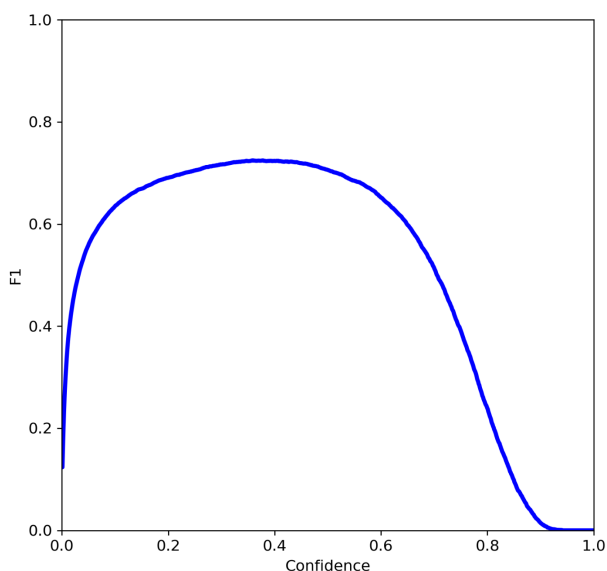


Fig. 8. F1 score against confidence threshold.

A sample detection output on a test image is shown in Fig. 9, demonstrating the model correctly identifying multiple parasites in a dense field of red blood cells.

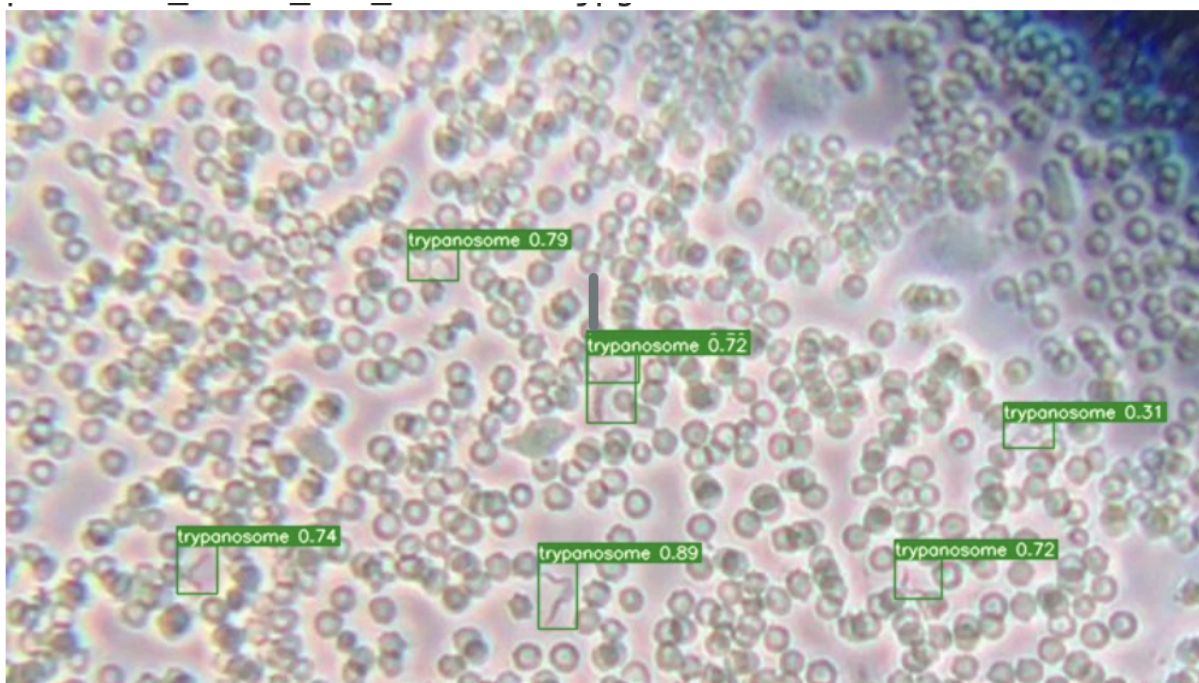


Fig. 9. Example detection output on a test image. Green bounding boxes indicate predicted parasite locations with associated confidence scores.

These results are competitive with published baselines on the Tryp dataset. Anzaku et al. [7] report 71.3% AP@0.5 for their YOLO-Tryppa architecture, which uses a modified detection

head with a dedicated P2 scale for small object detection. Our lightweight variant matches or exceeds this performance while using only about 6 million parameters — roughly 6× fewer than the full YOLOv7 architecture [4].

Confusion Matrix Analysis

The confusion matrix at the default operating threshold is shown in Fig. 10. Approximately 78% of actual parasites are correctly detected, with 22% misclassified as background (false negatives). The right column of the matrix, which reflects the format YOLOv7 uses for single-class detection, indicates where false positive predictions are categorized; since trypanosome is the only positive class, every false positive is labeled as such. This is consistent with the overall 74.9% precision and does not indicate an unusually high false positive rate.



Fig. 10. Confusion matrix on the test set. The top-left cell shows the rate of correctly detected parasites (78%), while the bottom-left shows the rate of missed parasites (22%).

Inference Speed and Model Size

After ONNX export, the model achieves the deployment characteristics summarized in Table III.

Table III. Model size and inference speed across deployment configurations.

Metric	Value
Model size (PyTorch)	~24 MB
Model size (ONNX)	~12 MB
GPU inference (T4)	6.5 ms/image

Metric	Value
CPU inference (ONNX Runtime)	~50-80 ms/image
Parameters	6,007,596
GFLOPs	13.0

The ONNX-exported model is roughly half the size of the original PyTorch checkpoint and can be distributed as a single 12 MB file. CPU inference via ONNX Runtime is fast enough for practical screening use — at roughly 50-80 ms per image, a clinician can process several hundred slides per hour without any GPU.

Training Convergence

Training and validation curves are shown in Fig. 11. Both precision and recall stabilized around epoch 50, with the model converging well within the 100-epoch training budget. This indicates that additional training would likely yield diminishing returns and that the model is well-fit rather than under-trained or overfit. The use of COCO-pretrained weights via transfer learning likely contributed to this rapid convergence.

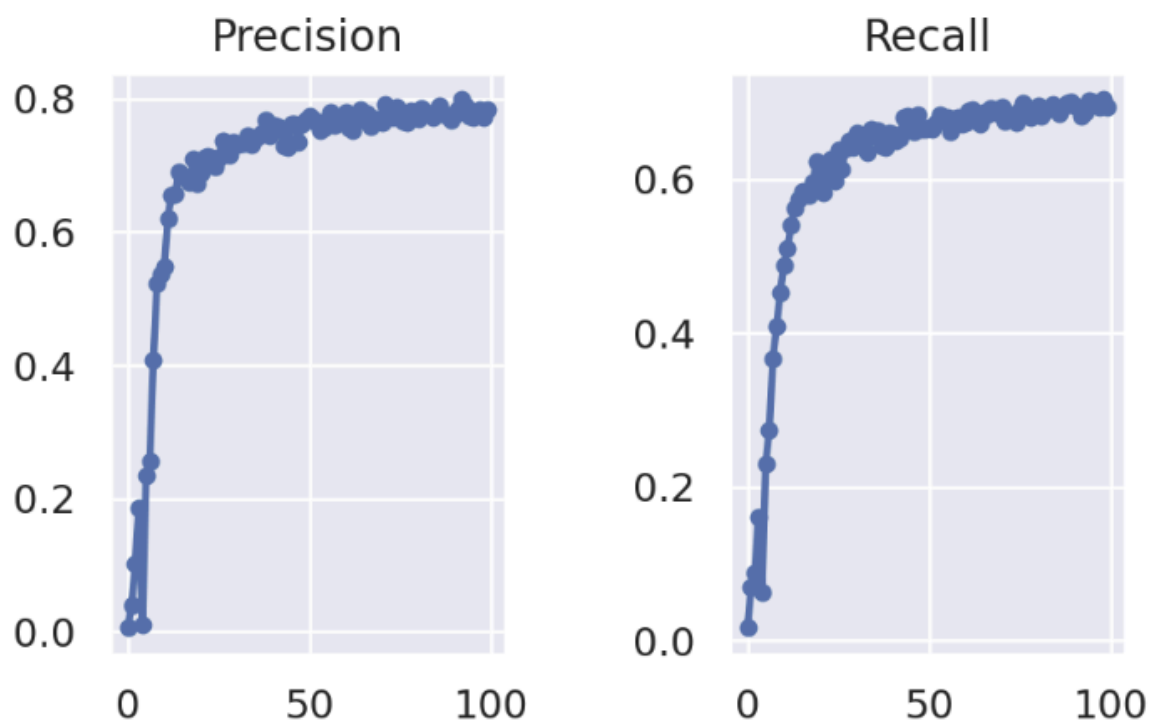


Fig. 11. Precision and recall over 100 training epochs. Both metrics stabilize around epoch 50, indicating convergence.

VI. Limitations

While the results demonstrate that lightweight trypanosome detection is feasible, the project has several limitations that should be acknowledged. The model was trained and evaluated exclusively on *Trypanosoma brucei brucei*. It cannot distinguish between different trypanosome species such as *T. cruzi* or *T. evansi*, which would be required for broader diagnostic applicability. Existing systems like Kittichai et al. [5] handle multi-species classification but at the cost of computational complexity.

The Tryp dataset also consists entirely of unstained thick blood smears. Performance on stained thin films, fluorescent imaging, or samples prepared using different protocols has not been validated. Real-world clinical samples vary significantly in preparation quality, and generalization to these conditions is unknown.

In addition, a recall of 70.2% means roughly 30% of parasites in the test set are missed by the model. This level of performance is appropriate for screening applications where flagged samples receive secondary expert review, but is insufficient for use as a standalone diagnostic tool. A missed positive in clinical practice would represent a missed diagnosis with potentially fatal consequences.

Finally, the system has been evaluated only on the held-out test split of the Tryp dataset. It has not been deployed or tested in actual clinical environments, on different microscope models, with varying lighting conditions, or with images captured by non-specialist users.

VII. Conclusion

This project demonstrates that competitive trypanosome detection is achievable using a lightweight model running on consumer hardware. By fine-tuning YOLOv7-tiny [4] from COCO-pretrained weights [13] on the publicly available Tryp dataset [6], the resulting model achieves 74.1% AP@0.5 — comparable to and in some metrics exceeding published baselines that rely on substantially larger architectures.

The key contribution of this work is not in the model architecture itself but in demonstrating an end-to-end accessible pipeline. Training was completed entirely on free Google Colab GPU resources. The resulting model was exported to ONNX format [8], producing a 12 MB file that runs on a standard laptop CPU using only three Python packages. A standalone command-line inference tool and a web-based Streamlit interface were both developed to demonstrate that the model can be deployed practically by users without machine learning expertise.

The broader significance is in deployment accessibility. Existing AI systems for trypanosome detection achieve high accuracy but require GPU hardware and proprietary platforms unavailable in the rural sub-Saharan African clinics where HAT screening is most needed. By prioritizing lightweight deployment, this work shows a viable path toward bringing automated

parasite screening to the settings where it would have the greatest public health impact — even at the cost of some accuracy compared to larger, GPU-bound alternatives.

While the model is not accurate enough to function as a standalone diagnostic, its role as an accessible first-line screening tool — flagging potentially positive samples for human confirmation — represents a meaningful improvement over the zero-coverage baseline that exists in clinics without trained microscopists.

VIII. Future Work and Recommendations

Several extensions could improve the system's accuracy and broaden its applicability. The most natural extension is to expand the model to distinguish between *T. brucei*, *T. cruzi*, and *T. evansi*. This would require integrating additional datasets with consistent annotation standards and would likely benefit from a hybrid detection-plus-classification architecture similar to Kittichai et al. [5], adapted to remain lightweight.

Although YOLO's grid-based detection naturally encounters background regions during training, explicitly including the Tryp dataset's 93 negative images during training could help further reduce false positives. This is particularly important for clinical use, where high specificity is essential to avoid unnecessary follow-up testing. Additionally, evaluating and fine-tuning the model on stained thin blood films would extend its applicability to the imaging modality used in most clinical laboratories. A small adaptation dataset combined with domain transfer techniques may suffice.

Some other future possibilities include mobile deployment, real-time inference from microscopy videos, and field validation. ONNX models can be further optimized using mobile-targeted formats such as TensorFlow Lite or CoreML. Packaging the detector as an Android or iOS application would dramatically increase its accessibility for field clinicians, allowing detection on smartphones connected to portable digital microscopes. The current pipeline also only operates on still images. Extending it to process live video feeds from digital microscopes would streamline the clinical workflow and reduce the steps required for screening. Ultimately, the system's clinical value can only be established through deployment trials in actual endemic-region clinics, evaluated against expert microscopy diagnoses. Such a study would also surface practical challenges related to image quality, user interface design, and integration with existing clinical workflows.

IX. Materials

- **Dataset:** Tryp dataset by Anzaku et al. [6], available at <https://doi.org/10.6084/m9.figshare.22825787>
- **Base model:** YOLOv7 by Wang et al. [4], available at <https://github.com/WongKinYiu/yolov7>
- **Pretrained weights:** YOLOv7-tiny COCO-pretrained weights ([yolov7-tiny.pt](#))
- **Compute:** Google Colab free tier with NVIDIA T4 GPU

- **Deployment hardware:** Standard laptop CPU (no GPU required)

Software and Packages

- Python 3.10+
- PyTorch (training)
- ONNX and ONNX Runtime [8] (model export and CPU inference)
- OpenCV (image processing)
- NumPy (numerical operations)
- Streamlit (web interface)
- Matplotlib (visualization)
- pycocotools (COCO format handling)

References

- [1] Centers for Disease Control and Prevention, "DPDx — Trypanosomiasis, American." [Online]. Available: <https://www.cdc.gov/dpdx/trypanosomiasisamerican/index.html>
- [2] P. P. Simarro, G. Cecchi, J. R. Franco, M. Paone, A. Diarra, J. A. Ruiz-Postigo, E. M. Fèvre, R. C. Mattioli, and J. G. Jannin, "Estimating and mapping the population at risk of sleeping sickness," *PLoS Negl. Trop. Dis.*, vol. 6, no. 10, p. e1859, 2012. doi: 10.1371/journal.pntd.0001859
- [3] Eisai Co., Ltd., "Diseases of Africa." [Online]. Available: <https://www.eisai.com/sustainability/people/globalhealth/diseases/africa.html>
- [4] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, "YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors," arXiv preprint arXiv:2207.02696, 2022. [Online]. Available: <https://arxiv.org/abs/2207.02696>
- [5] V. Kittichai, M. Kaewthamasorn, S. Thanee, T. Sasisaowapak, K. M. Naing, R. Jomtarak, T. Tongloy, S. Chuwongin, and S. Boonsang, "Superior auto-identification of trypanosome parasites by using a hybrid deep-learning model," *J. Vis. Exp.*, no. 200, 2023. doi: 10.3791/65557
- [6] E. T. Anzaku, M. A. Mohammed, U. Ozbulak, J. Won, H. Hong, J. Krishnamoorthy, S. Van Hoecke, S. Magez, A. Van Messem, and W. De Neve, "Tryp: A dataset of microscopy images of unstained thick blood smears for trypanosome detection," *Sci. Data*, vol. 10, p. 716, 2023. doi: 10.1038/s41597-023-02608-y
- [7] E. T. Anzaku et al., "YOLO-Tryppa: A modified YOLO architecture with improved small object detection for trypanosome parasite identification," 2025. [Online]. Available: <https://pmc.ncbi.nlm.nih.gov/articles/PMC12027583/>

[8] J. Bai, F. Lu, K. Zhang, et al., "ONNX: Open Neural Network Exchange," GitHub repository, 2019. [Online]. Available: <https://github.com/onnx/onnx>

[9] World Health Organization, "Trypanosomiasis, human African (sleeping sickness)." [Online]. Available: [https://www.who.int/news-room/fact-sheets/detail/trypanosomiasis-human-african-\(sleeping-sickness\)](https://www.who.int/news-room/fact-sheets/detail/trypanosomiasis-human-african-(sleeping-sickness))

[10] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, real-time object detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788. doi: 10.1109/CVPR.2016.91

[11] J. Solawetz, "YOLOv7 - A breakdown of how it works," Roboflow Blog, Jul. 17, 2022. <https://blog.roboflow.com/yolov7-breakdown/>

[12] C. Hughes, "YOLOv7: A deep dive into the current state-of-the-art for object detection | Towards Data Science," Towards Data Science, Nov. 25, 2022. <https://towardsdatascience.com/yolov7-a-deep-dive-into-the-current-state-of-the-art-for-object-detection-ce3ffedeeab/> (accessed Jun. 01, 2026).

[13] T.-Y. Lin et al., "Microsoft COCO: Common objects in context," in *Proc. European Conf. Computer Vision (ECCV)*, 2014, pp. 740–755. doi: 10.1007/978-3-319-10602-1_48

Appendix

A. Code

All source code for this project, including preprocessing scripts, training notebooks, evaluation notebooks, the standalone CLI inference tool, and the Streamlit web interface, is included in the [Github repository](#). See the included [README.md](#) for folder structure and run instructions.

B. Training Logs and Results

Complete training logs, generated evaluation plots (confusion matrix, F1 curve, precision-recall curve, training curves), and intermediate model checkpoints are included in the [tryp_yolov7_results/](#) directory of the code submission.